Deprecate `const`-qualifier on begin/end of views

Document #:	P4331R0
Date:	2025-01-13
Project:	Programming Language C++
Audience:	SG9

Reply-to:	Jonathan Müller (think-cell) <foonathan@jonathanmueller.dev>

Contents

§ 1 Abstract

Some views have a `const`-qualified `begin()` member function (e.g. `transform_view`), while some views do not have a `const`-qualified member function (e.g. `filter_view`, `split_view`). This behavior is confusing, as it allows beginners to write code that take an arbitrary range by reference to `const`. Such code works for almost all forward ranges, but not once `filter_view`/`split_view`/… is added into the mix, or input ranges. We propose deprecating the `const` qualifier to ensure that **no** view is `const`-iteratable and not adding it to all future views.

§ 2 Background

A `view` is a light-weight non-owning (mostly) cheaply copyable (mostly) range. Conceptually, a `view` is like a smart reference to a container: copying a `view` does not copy the container, a `const view` does not provide `const` element access, etc. — they have reference semantics. For types with reference semantics there are two levels on `const`: whether the reference itself is `const` and cannot be changed or whether the referenced data is `const`. This distinction matters, and just like with pointers, where we have `T const*` and `T* const`, the two should not be confused.

Right now, most views in the standard library have a `const` qualified `begin()` member function, as they just return an iterator without doing mutation. This follows the general philosophy of making things `const`-qualified whenever possible. There are two kinds of views that don't have it:

1. Input views where calling `begin()` permanently consumes some data (e.g. `std::generator`).
2. Forward views where calling `begin()` does some computation which is then cached to require the O(1) amortized complexity of `begin()` (e.g. `filter_view`, `split_view`, views wrapping `filter_view`/`split_view`/…).

§ 3 Motivation

§ 3.1 Common pitfalls for users

While the philosophy of making things `const`-qualified whenever possible is good in most situations, with views specifically it has created problems. Views look like containers but absolutely are not containers. The following code, while looking sensible at first, should be considered bad practice:

```
template <typename Rng>
void do_something(const Rng& rng)
{
    for (auto& x : rng)
    {
        …
    }
}
```

The user has `const`-qualified the reference, since they don't want to change the values of the range, only process it. This logic is sound for containers, which have value semantics, but not for views, which have reference semantics. For views, `x` is not `const`: the const-ness of a view is independent of the const-ness of the elements. Furthermore, this function breaks as soon as e.g. a `filter_view`/`split_view`/… is involved.

The correct version takes the `Rng` by forwarding reference and manually ensures it is `const` in the body:

```
template <typename Rng>
void do_something(Rng&& rng)
{
    for (const auto& x : rng)
    {
        …
    }
}
```

Thus, **no *generic* function that takes an *arbitrary* `const Rng&` is correct**. We should want to discourage people from writing code like that by having it fail earlier.

§ 3.2 Implementation complexity

If a view is never `const` iteratable, the implementation is easy: just provide a single `begin()`/`end()`. But if a view is `const` iteratable if the base view is `const` iteratable, you need to provide two overloads: one for `const` and one for non-`const`; the latter only conditionally available. This is more work than only providing a non-`const` qualified view object.

§ 3.3 What is the benefit, anyway?

What do you get from having a view that is `const` iteratable? The only benefit is that you can actually use a `const` view to do iteration.

But why would you want a `const` view? What does the `const` qualifier get you? Only two things:

1. You have a tool to prevent accidentally re-assigning your view object to point to different data.
2. You can share your view objects between threads and have a guarantee that there are no data races.

Crucially, it does not guarantee that the underlying elements are `const`!

For a view like `std::span` or `std::string_view`, the benefits make sense: They are vocabulary types that might appear in a global variable or in complex code where you would want to prevent accidental reassignment.

However, this is not really useful for a view adaptor from `std::ranges`:

1. Views can only be assigned to an object of the same type. For a view adaptor, this type depends on a lambda, so it actually requires some effort to even have a situation where you have two different views of the same type that you could potentially re-assign to each other. And even if you end up in a situation, e.g. by having a factory function that returns a view given some range and you call it with two different ranges of the same type, why is there code that even attempts to do

assignment of views? Views are meant to be created on the fly, to iterate over some code, not stored in variables.

2. What is the use case for sharing views between threads? Views are cheap to construct, so you can always share your data and create separate view objects in each thread.

We argue that allowing `const` qualified view adaptors has only marginal benefits for increased implementation complexity. The main feature of `const` qualified view adaptors is enabling a pitfall for beginners.

§ 4 Prior discussion

The need for `const`-qualified `begin()`/`end()` on views has been discussed back in 2016 in [range-v3] [range-v3-discussion]. It raised similar points as in this paper:

Casey Carter:

> Views in range-v3 may have both const and non-const overloads of begin/end/size (herein termed “operations”). Views have pointer semantics - a view is essentially a pointer to a sequence of elements - so mutability of the elements viewed is orthogonal to mutability of the view object itself. The const distinction here has no relation to that of containers. Non-const operations do not modify the semantic objects being viewed, nor do they “swing the pointer” so that the same view designates different semantic objects. Non-const operations mutate internal state that does not contribute to the semantic value of the view; the const-ness here is purely bitwise.
>
> The const-ness model used by views makes view composition painful. […] I see a potential for latent bugs where a programmer accustomed to the fact that calling begin/end on mutable containers is threadsafe calls begin/end on mutable ranges without realizing there are sharp corners here. […]

Eric Niebler:

> One simplification is that no views have const begin()/end() members. That way nobody has to think about it. Seems like a fool’s consistency to me, though.

Gonzalo Brito Gadeschi:

> The root of the problem is that newcomers try to make views consts or try to pass them by const lvalue reference because they have an incomplete mental model of how views work.

Eric Niebler

> Although he originally said it in jest, I think that ericniebler's suggestion upthread that maybe NO views should be const iterable may be the best solution to user confusion about which views are const iterable and why.

I wasn't joking when I said that. It's still an attractive option.

Ultimately, a lot of the discussion focused around the need for caching in `begin()`, whether that should be `mutable` and locking, or non-`const`, that `Rng&&` is really the best way to pass in a view and not `const Rng&`, and the reference semantics of views. In the end, the design where caching was done in a non-const qualified `begin()` was chosen, and other views are `const`-iteratable whenever possible. The idea of omitting `const` was not seriously investigated, as far as we can tell.

§ 5 Proposal

We propose deprecating the `const` qualifier on `begin()`/`end()` of the standard library views (and consequently also on `empty()`, `size()` and all other member functions that you'd get by inheriting from `std::ranges::view_interface`). This deprecation affects code in two situations:

1. Non-generic/non-type-inferred code that have a `const view&`, where `view` is the concrete type of a `view`. It will call the `const` qualified member functions, which will be deprecated. However, we only propose deprecating it for the `std::views` range adaptors, where the name of the type is long and involves lambda, so not a lot of code exists.
2. Generic code/type inferred code that uses `const auto&`. However, such code would be broken as soon as a type like `filter_view`/`split_view`/… is involved anyway, and breaking it earlier is the primary motivation of this paper.

We also propose only deprecation and not removal, and implementations are free to deal with the deprecation in an appropriate way.

Concretely, we consider the following set of `const` qualified overloads of member functions (when present):

— `begin()`

— `end()`

— `empty()`

— `cbegin()`

— `cend()`

— `operator bool()`

— `data()`

— `size()`

— `front()`

— `back()`

— `operator[]`

And we propose deprecating it on the following standard library types:

— `std::ranges::ref_view`

— `std::ranges::owning_view`

— `std::ranges::as_rvalue_view`

— `std::ranges::transform_view`

— `std::ranges::take_view`

— `std::ranges::take_while_view`

— `std::ranges::drop_view`

— `std::ranges::drop_while_view`

— `std::ranges::join_view`

— `std::ranges::join_with_view`

— `std::ranges::lazy_split_view`

— `std::ranges::common_view`

— `std::ranges::reverse_view`

— `std::ranges::as_const_view`

— `std::ranges::elements_view`

— `std::ranges::enumerate_view`

— `std::ranges::zip_view`

— `std::ranges::zip_transform_view`

— `std::ranges::adjacent_view`

— `std::ranges::adjacent_transform_view`

— `std::ranges::chunk_view`

— `std::ranges::slide_view`

— `std::ranges::cartesian_product_view`

— `std::ranges::concat_view`

We do not propose deprecating or removing it on the following standard library types:

— `std::span`/`std::string_view` (it is a widely used, potentially re-assigned types where we would break code)

— `std::ranges::view_interface` (used by views below)

— `std::ranges::iota_view` (it is a factory with a reasonable type name)

— `std::ranges::repeat_view` (it is a factory with a reasonable type name)

— `std::ranges::empty_view` (the member functions are `static`)

— `std::ranges::single_view` (it actually has deep `const`)

Note that the following standard library types are views without `const` qualified versions of those member functions to begin with. As soon as code uses them with a `const` qualifier, it will break:

— `std::ranges::istream_view`

— `std::ranges::filter_view`

— `std::ranges::split_view`

— `std::ranges::chunk_by_view`

— `std::ranges::stride_view`

— `std::generator`

If a standard library types models `view` and is not mentioned in any list, we have forgotten about it.

§ 6 Wording

TBD

§ 7 References

[range-v3] range-v3.
https://github.com/ericniebler/range-v3
[range-v3-discussion] const-ness of view operations.
https://github.com/ericniebler/range-v3/issues/385